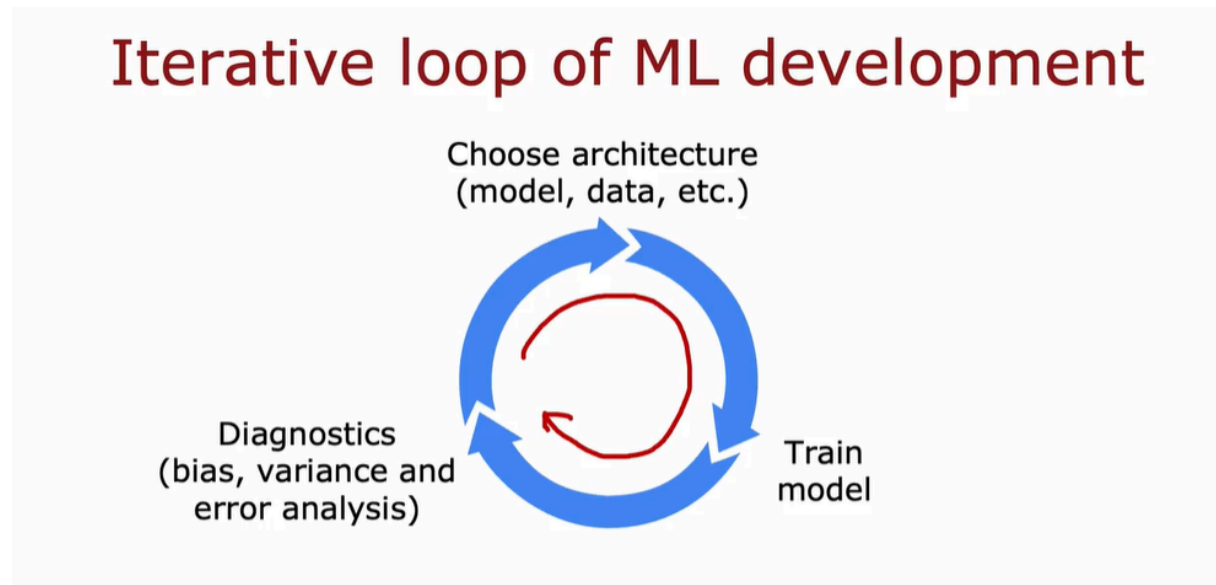


Machine Learning development process

Iterative loop of ML development



Example:

Spam classification example

From: cheapsales@buystufffromme.com To: Andrew Ng Subject: Buy now! Deal of the week! Buy now! Rolex <u>w4tchs</u> - \$100 <u>Medlcine</u> (any kind) - £50 Also low cost <u>M0rgages</u> available.	From: Alfred Ng To: Andrew Ng Subject: Christmas dates? Hey Andrew, Was talking to Mom about plans for Xmas. When do you get off work. Meet Dec 22? Alf
--	---

The email in the right is spam

Building a spam classifier

Supervised learning: $\vec{x}$ = features of email

y = spam (1) or not spam (0)

Features: list the top 10,000 words to compute $x_1, x_2, \dots, x_{10,000}$

$$\vec{x} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 1 \\ 0 \\ \vdots \end{bmatrix} \begin{matrix} a \\ andrew \\ buy \\ deal \\ discount \\ \vdots \end{matrix}$$

From: cheapsales@buystufffromme.com
To: Andrew Ng
Subject: Buy now!

Deal of the week! Buy now!
Rolex w4tchs - \$100
Medlcine (any kind) - £50
Also low cost M0rgages
available.

This application is an example of text classification because you're taking a text document that is an email and trying to classify it as either spam or non-spam.

One way to construct the features of the email would be to say, take the top 10,000 words in the English language or in some other dictionary and use them to define features x_1, x_2 through $x_{10,000}$.

For example, given this email on the right, if the list of words we have is "a", "Andrew" "buy" "deal" "discount" and so on.

Then given the email on the right, we would set these features to be, say, 0 or 1, depending on whether or not that word appears. The word a does not appear. The word Andrew does appear. The word buy does appear, deal does, discount does not, and so on, and so you can construct 10,000 features of this email.

There are many ways to construct a feature vector. **Another way would be to let these numbers not just be 1 or 0, but actually, count the number of times a given word appears in the email.** If buy appears twice, maybe you want to set this to 2, but setting into just 1 or 0. It actually works decently well. Given these features, you can then train a classification algorithm such as a logistic regression model or a neural network to predict y given these features x .

Building a spam classifier

How to try to reduce your spam classifier's error?

- Collect more data. E.g., "Honeypot" project. 
- Develop sophisticated features based on email routing (from email header).
- Define sophisticated features from email body. E.g., should "discounting" and "discount" be treated as the same word.
- Design algorithms to detect misspellings. E.g., w4tches, med1cine, m0rtgage.

Error analysis

Error analysis

m_{cv} = 500 examples in cross validation set.

Algorithm misclassifies 100 of them.

Manually examine 100 examples and categorize them based on common traits.

Pharma:

Deliberate misspellings (w4tches, med1cine):

Unusual email routing:

Steal passwords (phishing):

Spam message in embedded image:

Error analysis

$m_{cv} =$ ~~500~~⁵⁰⁰⁰ examples in cross validation set.

Algorithm misclassifies ~~100~~¹⁰⁰⁰ of them.

Manually examine 100 examples and categorize them based on common traits.

- Pharma: 21 more data features
- Deliberate misspellings (w4tches, med1cine): 3
- Unusual email routing: 7
- Steal passwords (phishing): 18 more data features
- Spam message in embedded image: 5

For example, there can be a pharmaceutical spam email that also has unusual routing or a password that has deliberate misspellings and is also trying to carry out the phishing attack.

One email can be counted in multiple categories. In this example, I had said that the algorithm misclassified as 100 examples and we'll look at all 100 examples manually. If you have a larger cross validation set, say we had 5,000 cross validation examples and if the algorithm misclassified say 1,000 of them then you may not have the time depending on the team size and how much time you have to work on this project. You may not have the time to manually look at all 1,000 examples that the algorithm misclassifies.

In that case, I will often sample randomly a subset of usually around a 100, maybe a couple 100 examples because that's the amount that you can look through in a reasonable amount of time. Hopefully looking through maybe around a 100 examples will give you enough statistics about whether the most common types of errors and therefore where maybe most fruitful to focus your attention. After this analysis, if you find that a lot of errors are pharmaceutical spam emails then this might give you some ideas or inspiration for things to do next. For example, you may decide to collect more data but not more data of everything, but just try to find more data of pharmaceutical spam emails so that the learning algorithm can do a better job recognizing these pharmaceutical spam.

Or you may decide to come up with some new features that are related to say specific names of drugs or specific names of pharmaceutical products of the spammers are trying to sell in order to help your learning algorithm become

better at recognizing this type of pharma spam. Then again this might inspire you to make specific changes to the algorithm relating to detecting phishing emails. For example, you might look at the URLs in the email and write special code to come with extra features to see if it's linking to suspicious URLs.

Or again, you might decide to get more data of phishing emails specifically in order to help your learning algorithm do a better job of recognizing them. The point of this error analysis is by manually examining a set of examples that your algorithm is misclassifying or mislabeling. Often this will create inspiration for what might be useful to try next and sometimes it can also tell you that certain types of errors are sufficiently rare that they aren't worth as much of your time to try to fix.

Adding data

Adding data

Add more data of everything. E.g., "Honeypot" project.

Add more data of the types where error analysis has indicated it might help.

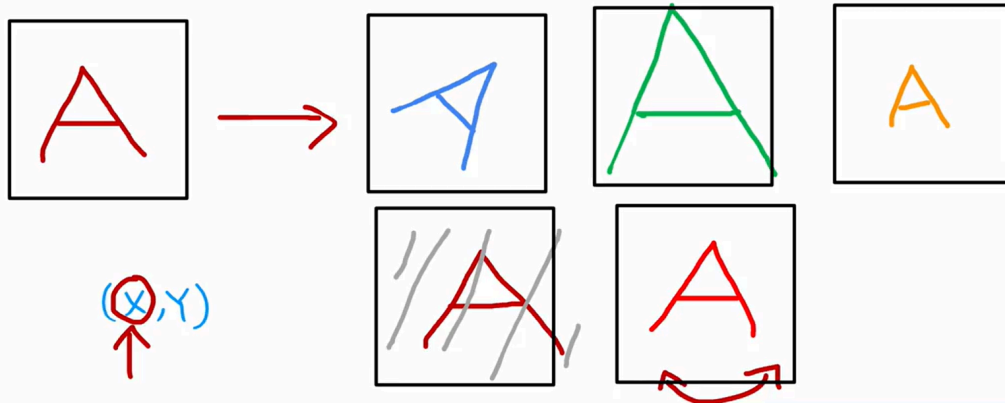
Pharma spam
E.g., Go to unlabeled data and find more examples of Pharma related spam.

Beyond getting brand new training examples (x,y), another technique: Data augmentation

Data aumentation

Data augmentation

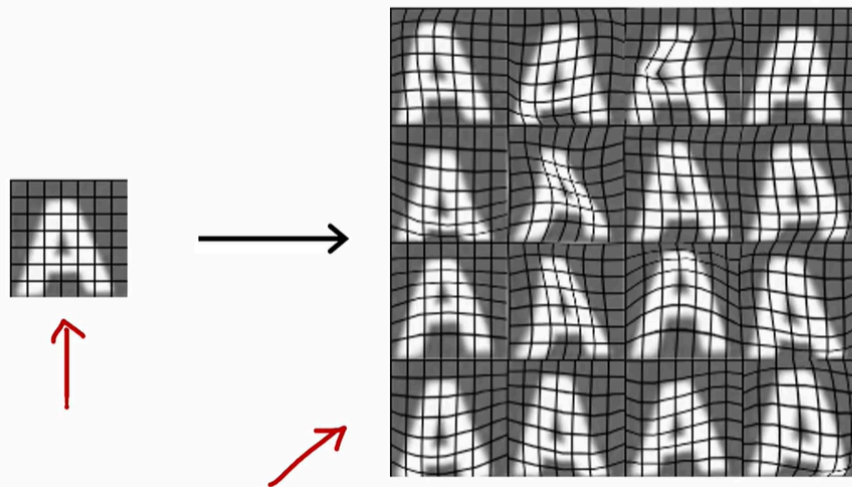
Augmentation: modifying an existing training example to create a new training example.



For example if you're trying to recognize the letters from A to Z for an OCR optical character recognition problem.

- So not just the digits 0-9 but also the letters from A to Z. Given an image like this, you might decide to create a new training example by **rotating the image a bit**. Or by **enlarging the image a bit** or by **shrinking a little bit** or by **changing the contrast of the image**.
- And these are examples of distortions to the image that don't change the fact that this is still the letter A. And for some letters but not others you can also **take the mirror image** of the letter and it still looks like the letter A. But this only applies to some letters but these would be ways of taking a training example X, Y.
- And applying a distortion or transformation to the input X, in order to come up with another example that has the same label. And by doing this you're telling the algorithm that the letter A rotated a bit or enlarged a bit or shrunk a little bit it is still the letter A.
- And creating additional examples like this helps the learning algorithm, do a better job learning how to recognize the letter A.

Data augmentation by introducing distortions



For a more advanced example of data augmentation. You can also take the letter A and place a grid on top of it. And by introducing random warping of this grid, you can take the letter A.

And introduce warpings of the leather A to create a much richer library of examples of the letter A. And this process of distorting these examples then has turned one image of one example into here training examples that you can feed to the learning algorithm to hope it learn more robustly.

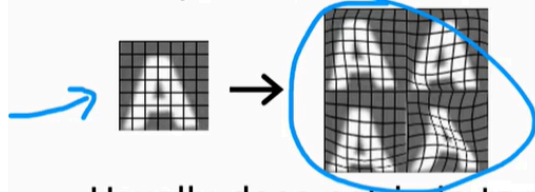
Data augmentation for speech

Speech recognition example

-  Original audio (voice search: "What is today's weather?")
-  + Noisy background: Crowd
-  + Noisy background: Car
-  + Audio on bad cellphone connection

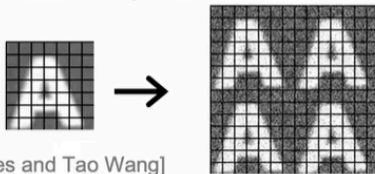
Data augmentation by introducing distortions

Distortion introduced should be representation of the type of noise/distortions in the test set.



Audio:
Background noise,
bad cellphone connection

Usually does not help to add purely random/meaningless noise to your data.



x_i = intensity (brightness) of pixel i
 $x_i \leftarrow x_i + \text{random noise}$

Adam Coates and Tao Wang]

And so we've seen how you can take one audio clip and turn it into three training examples here, one with crowd background noise, one with car background noise and one as if it was recorded on a bad cell phone connection. And the times I worked on speech recognition systems, this was actually a really critical technique for increasing artificially the size of the training data I had to build a more accurate speech recognizer.

One tip for data augmentation is that the changes or the distortions you make to the data, should be representative of the types of noise or distortions in the test set. So for example, if you take the letter a and warp it like this, this still looks like examples of letters you might see out there that you would like to recognize. Or for audio adding background noise or bad cellphone connection if that's representative of what you expect to hear in the test set, then this will be helpful ways to carry out data augmentation on your audio data.

In contrast is usually not that helpful at purely random meaningless noise to data. For example, you have taken the letter A and I've added per pixel noise where if x_i is the intensity or the brightness of pixel i , if I were to just add noise to each pixel, they end up with images that look like this. But if to the extent that this isn't that representative of what you see in the test set because you don't often get images like this in the test set is actually going to be less helpful.

So one way to think about data augmentation is how can you modify or warp or distort or make more noise in your data. But in a way so that what you get is still quite similar to what you have in your test set, because that's what the learning algorithm will ultimately end up doing well on. Now, whereas data augmentation

takes an existing training example and modifies it to create another training example.

Artificial data synthesis for photo OCR



There's one of the techniques which is data synthesis in which you make up brand new examples from scratch. Not by modifying an existing example but by creating brand new examples. So take the example of photo OCR. Photo OCR or photo optical character recognition refers to the problem of looking at an image like this and automatically having a computer read the text that appears in this image. So there's a lot of text in this image. How can you train an OCR algorithm to read text from an image like this? Well, when you look closely at what the letters in this image look like they actually look like this. So this is real data from a photo OCR task.

It can be a lot of work to write the code to generate realistic looking synthetic data for a given application. But when you spend the time to do so, it can sometimes help you generate a very large amount of data for your application and give you a huge boost to your algorithm's performance.

Artificial data synthesis for photo OCR



Real data



Synthetic data

[Adam Coates and Tao Wang]

Synthetic data generation has been used most probably for computer vision tasks and less for other applications. Not that much for audio tasks as well. All the techniques you've seen in this video related to finding ways to engineer the data used by your system. In the way that machine learning has developed over the last several decades, many decades.

Engineering the data used by your system

Conventional
model-centric
approach:

$$\text{AI} = \text{Code} + \text{Data}$$

(algorithm/model)

work on this

Data-centric
approach:

$$\text{AI} = \text{Code} + \text{Data}$$

(algorithm/model)

work on this

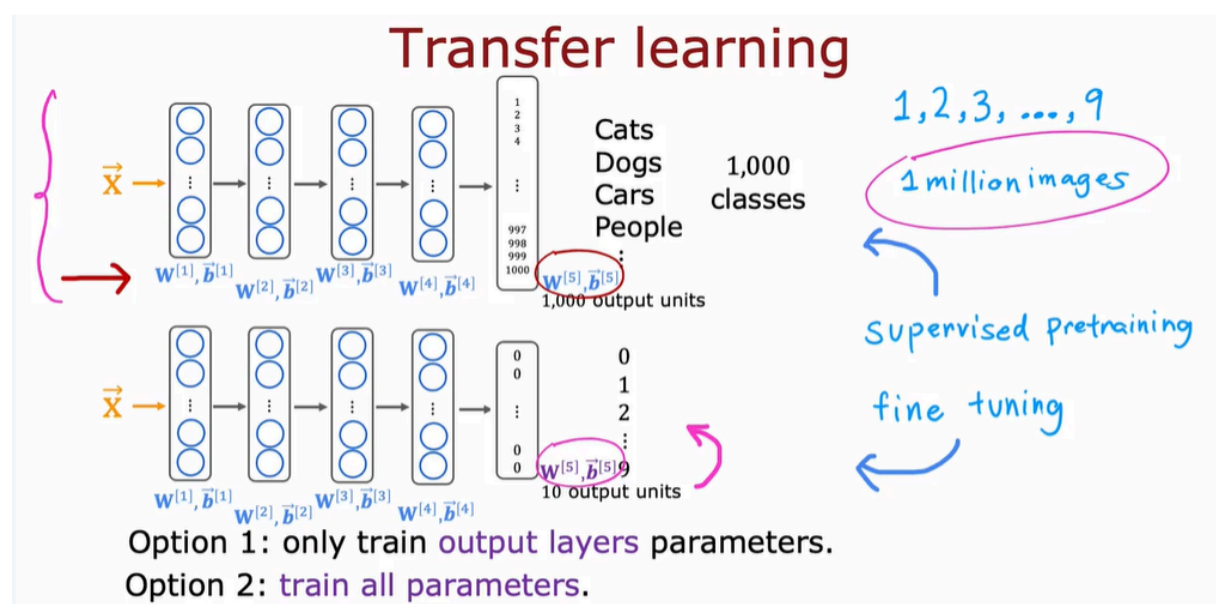
most researchers doing machine learning research would download the data set and hold the data fixed while they focus on improving the code or the algorithm or the model.

Thanks to that paradigm of machine learning research. I find that today the algorithm we have access to such as linear regression, logistic regression,

neural networks, also decision trees we should see next week. There are algorithms that already very good and will work well for many applications. And so sometimes it can be more fruitful to spend more of your time taking a data centric approach in which you focus on engineering the data used by your algorithm. And this can be anything from collecting more data just on pharmaceutical spam. If that's what error analysis told you to do. To using data augmentation to generate more images or more audio or using data synthesis to just create more training examples. And sometimes that focus on the data can be an efficient way to help your learning algorithm improve its performance.

Transfer learning: using data from a different task

For an application where you don't have that much data, transfer learning is a wonderful technique that lets you use data from a different task to help on your application. This is one of those techniques that I use very frequently.



Let's take a look at how transfer learning works. Here's how transfer learning works. Let's say you want to recognize the handwritten digits from zero through nine but you don't have that much labeled data of these handwritten digits. Here's what you can do.

Say you find a very large datasets of one million images of pictures of cats, dogs, cars, people, and so on, a thousand classes. You can then start by

training a neural network on this large dataset of a million images with a thousand different classes and train the algorithm to take as input an image X , and learn to recognize any of these 1,000 different classes. In this process, you end up learning parameters for the first layer of the neural network W^1, b^1 , for the second layer W^2, b^2 , and so on, W^3, b^3, W^4, b^4 , and W^5, b^5 for the output layer.

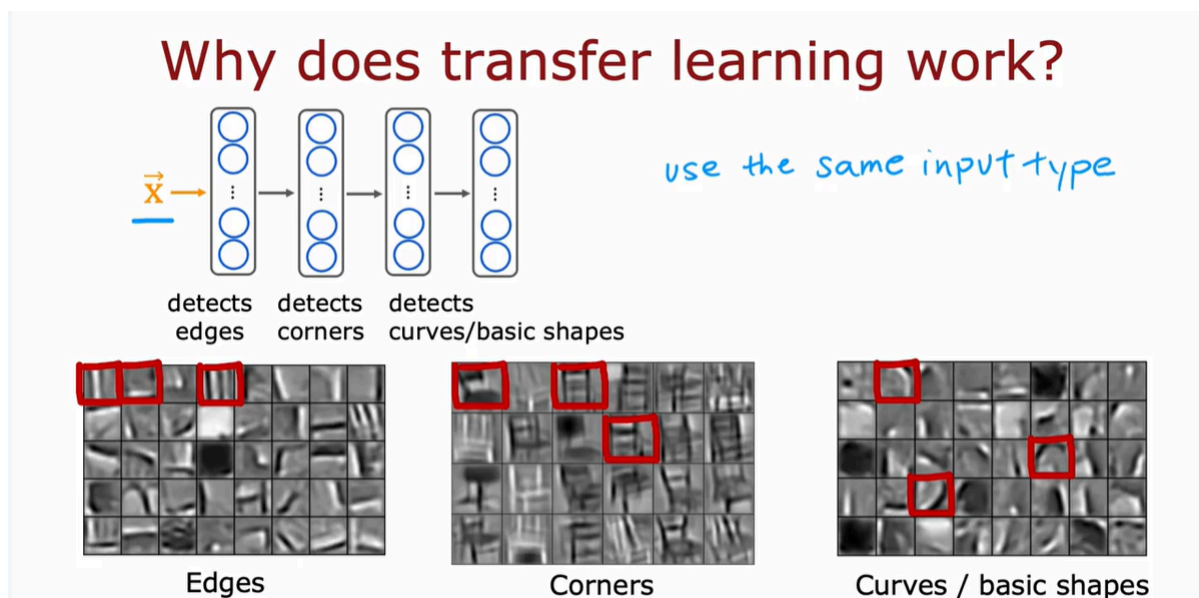
To apply transfer learning, what you do is then **make a copy of this neural network** where you would keep the parameters $W^1, b^1, W^2, b^2, W^3, b^3$, and W^4, b^4 .

- But for the last layer, you would eliminate the output layer and replace it with a much smaller output layer with just 10 rather than 1,000 output units. These 10 output units will correspond to the classes zero, one, through nine that you want your neural network to recognize.
 - Notice that the parameters W^5, b^5 they can't be copied over because the dimension of this layer has changed, so you need to come up with new parameters W^5, b^5 that you need to train from scratch rather than just copy it from the previous neural network.
- In transfer learning, what you can do is use the parameters from the first four layers, really all the layers except the final output layer as a starting point for the parameters and then run an optimization algorithm such as gradient descent or the Adam optimization algorithm with the parameters initialized using the values from this neural network up on top.
- In detail, there are two options for how you can train this neural networks parameters.
- Option 1 is you only train the output layers parameters. You would take the parameters W^1, b^1, W^2, b^2 through W^4, b^4 as the values from on top and just hold them fix and don't even bother to change them, and use an algorithm like Stochastic gradient descent or the Adam optimization algorithm to only update W^5, b^5 to lower the usual cost function that you use for learning to recognize these digits zero to nine from a small training set of these digits zero to nine, so this is Option 1.
- Option 2 would be to train all the parameters in the network including W^1, b^1, W^2, b^2 all the way through W^5, b^5 but the first four layers parameters would be initialized using the values that you had trained on top. If you have a very small training set then Option 1 might work a little bit

better, but if you have a training set that's a little bit larger then Option 2 might work a little bit better.

- This algorithm is called transfer learning because the intuition is by learning to recognize cats, dogs, cows, people, and so on. It will hopefully, have learned some plausible sets of parameters for the earlier layers for processing image inputs. Then by transferring these parameters to the new neural network, the new neural network starts off with the parameters in a much better place so that we have just a little bit of further learning. Hopefully, it can end up at a pretty good model. **These two steps of first training on a large dataset and then tuning the parameters further on a smaller dataset go by the name of supervised pre-training for this step on top.**
- That's when you train the neural network on a very large dataset of say a million images of not quite the related task. Then the second step is called **fine tuning** where you take the parameters that you had initialized or gotten from supervised pre-training and then run gradient descent further to fine tune the weights to suit the specific application of handwritten digit recognition that you may have. If you have a small dataset, even tens or hundreds or thousands or just tens of thousands of images of the handwritten digits, being able to learn from these million images of a not quite related task can actually help your learning algorithm's performance a lot. One nice thing about transfer learning as well is maybe you don't need to be the one to carry out supervised pre-training.
- For a lot of neural networks, there will already be researchers they have already trained a neural network on a large image and will have posted a trained neural networks on the Internet, freely licensed for anyone to download and use. What that means is rather than carrying out the first step yourself, you can just download the neural network that someone else may have spent weeks training and then replace the output layer with your own output layer and carry out either Option 1 or Option 2 to fine tune a neural network that someone else has already carried out supervised pre-training on, and just do a little bit of fine tuning to quickly be able to get a neural network that performs well on your task. Downloading a pre-trained model that someone else has trained and provided for free is one of those techniques where by building on each other's work on machine learning community we can all get much better results. By the generosity of other researchers that have pre-trained and posted their neural networks online.

But why does transfer learning even work? How can you possibly take parameters obtained by recognizing cats, dogs, cars, and people and use that to help you recognize something as different as handwritten digits?



That's why by training a neural network to detect things as diverse as cats, dogs, cars and people, you're helping it to learn to detect these pretty generic features of images and finding edges, corners, curves, basic shapes. This is useful for many other computer vision tasks, such as recognizing handwritten digits.

One restriction of pre-training though, is that the image type x has to be the same for the pre-training and fine-tuning steps. If the final task you want to solve is a computer vision tasks, then the pre-training step also has been a neural network trained on the same type of input, namely an image of the desired dimensions.

Conversely, if your goal is to build a speech recognition system to process audio, then a neural network pre-trained on images probably won't do much good on audio. Instead, you want a neural network pre-trained on audio data, there you then fine tune on your own audio dataset and the same for other types of applications. You can pre-train a neural network on text data and if your application has a save feature input x of text data, then you can fine tune that neural network on your own data.

Transfer learning summary

1. Download neural network parameters pretrained on a large dataset with same input type (e.g., images, audio, text) as your application (or train your own).
2. Further train (fine tune) the network on your own data.

1 million images

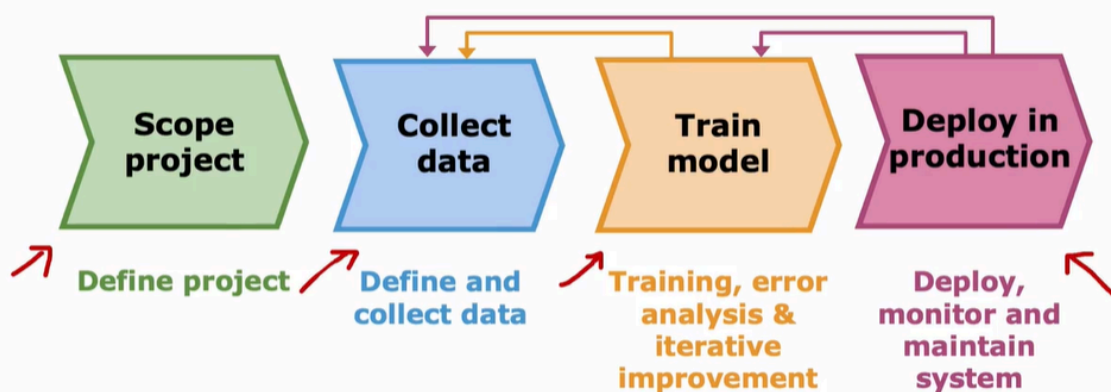
1000 images

50 images

Full cycle of a machine learning project

But when I'm building a machine learning system I find that training a model is just part of the puzzle. In this video I'd like to share with you what I think of as the full cycle of a machine learning project. That is, when you're building a valuable machine learning system, what are the steps to think about and plan for?

Full cycle of a machine learning project



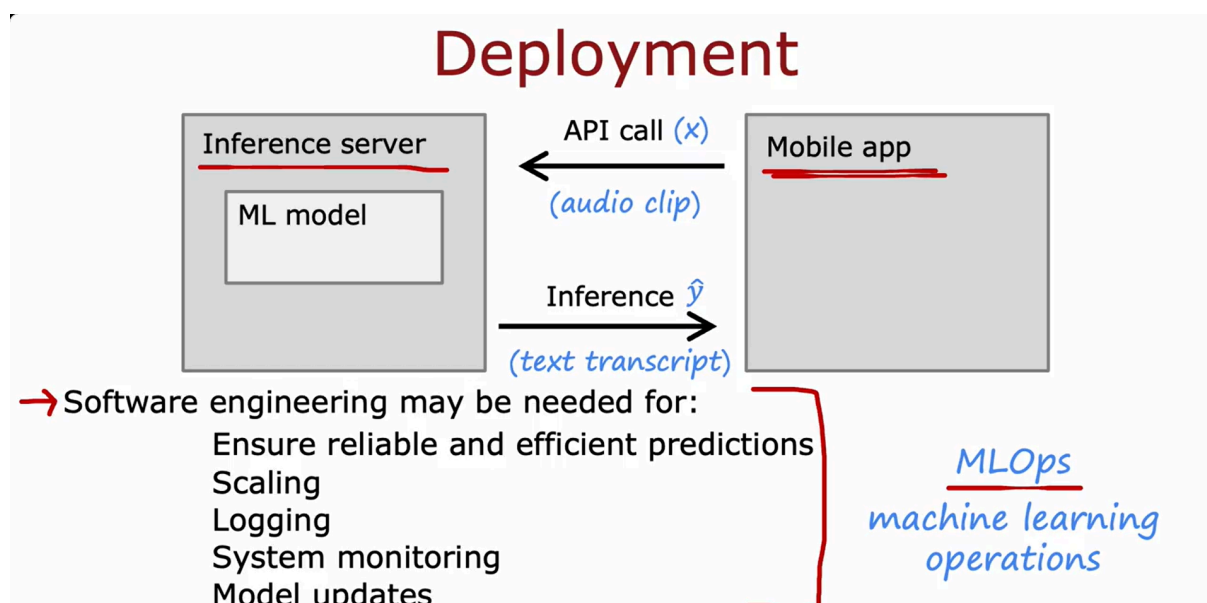
Let's take a look, let me use speech recognition as an example to illustrate the full cycle of a machine learning project.

- The first step of machine learning project is to scope the project. In other words, decide what is the project and what you want to work on. For

example, I once decided to work on speech recognition for voice search. That is to do web search using speaking to your mobile phone rather than typing into your mobile phone. This project scoping.

- After deciding what to work on you have to collect data. Decide what data you need to train your machine learning system and go and do the work to get the audio and get the transcripts of the labels for your dataset. That's data collection. After you have your initial data collection you can then start to train the model.
- Train the model. Here you would train a speech recognition system and carry out error analysis and iteratively improve your model. Is not at all uncommon. After you've started training the model for error analysis or for a bias-variance analysis to tell you that you might want to go back to collect more data. Maybe collect more data of everything or just collect more data of a specific type where your error analysis tells you you want to improve the performance of your learning algorithm. For example, once when working on speech I realized that my model was doing particularly poorly when there was car noise in the background. That sounded like someone was speaking in a car. My speech system perform poorly decided to get more data, actually using data augmentation to get more speech data that sounds like it was a car in order to improve the performance of my learning algorithm. You go around this loop a few times, train the model, error analysis, go back to collect more data, maybe do this for a while until eventually you say the model is good enough to then deploy in a production environment.
- Deploy in a production environment. What that means is you make it available for users to use. When you deploy a system you have to also make sure that you continue to monitor the performance of the system and to maintain the system in case the performance gets worse to bring us performance back up instead of just hosting your machine learning model on a server. I'll say a little bit more about why you need to maintain these machine learning systems on the next slide. But after this deployment, sometimes you realize that is not working as well as you hoped and you go back to train the model to improve it again or even go back and get more data. In fact, if users and if you have permission to use data from your production deployment, sometimes that data from your working speech system can give you access to even more data with which to keep on improving the performance of your system. Now, I think you have a sense

of what scoping a project means and we've talked a bunch about collecting data and training models in this course.



But let me share with you a little bit more detail about what deploying in production might look like. After you've trained a high performing machine learning model, say a speech recognition model, a common way to deploy the model would be to take your machine learning model and implement it in a server, which I'm going to call an inference server, whose job it is to call your machine learning model, your trained model, in order to make predictions.

Then if your team has implemented a mobile app, say a search application, then when a user talks to the mobile app, the mobile app can then make an API call to pass to your inference server the audio clip that was recorded and the inference server's job is supply the machine learning model to it and then return to it the prediction of your model, which in this case would be the text transcripts of what was said. This would be a common way of implementing an application that calls via the API and inference server that has your model repeatedly make predictions based on the input, x . This were common pattern where depend on the application does implemented. You have an API call to give your learning algorithm the input, x , and your machine learning model within output to prediction, say $\hat{y}$. To implement this some software engineering may be needed to write all the code that does all of these things.

Depending on whether your application needs to serve just a few handful of users or millions of users the amounts of software engineer needed can be quite different. I've build software that serve just a handful of users on my

laptop and I've also built software that serves hundreds of millions of users requiring significant data center resources. Depending on scale application needed, software engineering may be needed to make sure that your inference server is able to make reliable and efficient predictions hopefully not too high of computational cost. Software engineering may be needed to manage scaling to a large number of users. You often want to log the data you're getting both the inputs, x , as well as the predictions, $\hat{y}$, assuming that user privacy and consent allows you to store this data.

This data, if you can access to it, is also very useful for system monitoring. For example, I once built a speech recognition system on a certain dataset that I had but when there were new celebrities that suddenly became well-known or elections cause new politicians to become elected and people will search for these new names that were not in the training set and then my system did poorly on. It was because we were monitoring the system allowed us to figure out when the data was shifting and the algorithm was becoming less accurate. This allowed us to retrain the model and then to carry out a model update to replace the old model with a new one. The deployment process can require some amounts of software engineering.

For some applications, if you're just running it on a laptop or on a one or two servers, maybe not that much software engineering is needed. Depending on the team you're working on, it is possible that you built the machine learning model but there could be a different team responsible for deploying it. But there is a growing field in machine learning called MLOps. This stands for Machine Learning Operations. This refers to the practice of how to systematically build and deploy and maintain machine learning systems.

To do all of these things to make sure that your machine learning model is reliable, scales well, has good laws, is monitored, and then you have the opportunity to make updates to the model as appropriate to keep it running well. For example, if you are deploying your system to millions of people you may want to make sure you have a highly optimized implementations so that the compute cost of serving millions of people is not too expensive. In this and the last class I spent a lot of time talking about how to train a machine learning model and got this absolutely the critical piece to making sure you have a high performance system. If you ever have to deploy system to millions of people, these are some additional steps that you probably have to address.

Fairness, bias, and ethics

Bias

Hiring tool that discriminates against women.

Facial recognition system matching dark skinned individuals to criminal mugshots.

Biased bank loan approvals.

Toxic effect of reinforcing negative stereotypes.

Adverse use cases

Deepfakes

Spreading toxic/incendiary speech through optimizing for engagement.

Generating fake content for commercial or political purposes.

Using ML to build harmful products, commit fraud etc.

Spam vs anti-spam : fraud vs anti-fraud.

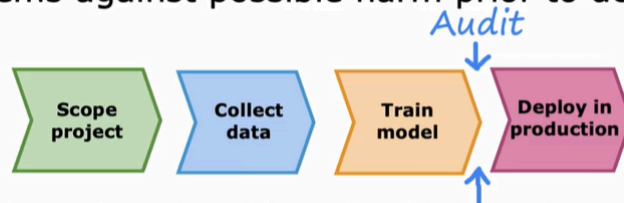
Ethics is a very complicated and very rich subject that humanity has studied for at least a few 1000 years. When AI became more widespread, I actually went and read up multiple books on philosophy and multiple books on ethics because I was hoping naively it turned out to come up with if only there's a checklist of five things we could do and so as we do these five things then we can be ethical, but I failed And I don't think anyone has ever managed to come up with a simple checklist of things to do to give that level of concrete guidance about how to be ethical.

Guidelines

Get a diverse team to brainstorm things that might go wrong, with emphasis on possible harm to vulnerable groups.

Carry out literature search on standards/guidelines for your industry.

Audit systems against possible harm prior to deployment.



Develop mitigation plan (if applicable), and after deployment, monitor for possible harm.